| | |
|---|---|
| **Status** | Finished |
| **Started** | Monday, 28 September 2026, 8:42 AM |
| **Completed** | Monday, 28 September 2026, 8:45 AM |
| **Duration** | 2 mins 34 secs |
| **Grade** | **10.00** out of 10.00 (**100**%) |

Question **1**

Correct

Mark 2.00 out of 2.00

complete function to implement coin change making problem i.e. finding the minimum

number of coins of certain denominations that add up to given amount of money.

The only available coins are of values 1, 2, 3, 4

Input Format:

Integer input from stdin.

Output Format:

return the minimum number of coins required to meet the given target.

Example Input:

16

Output:

4

Explanation:

We need only 4 coins of value 4 each

Example Input:

25

Output:

7

Explanation:

We need 6 coins of 4 value, and 1 coin of 1 value

**Answer:** (penalty regime: 0 %)

Reset answer

```
def coinChange(amount: int) -> int:
    if amount <= 0:
        return 0

    # Take as many 4-value coins as possible
    coins_of_four = amount // 4
    remainder = amount % 4

    # Any remainder (1, 2, or 3) can be fulfilled with exactly 1 coin
    if remainder > 0:
        return coins_of_four + 1

    return coins_of_four

```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | print(coinChange(16)) | 4 | 4 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 2.00/2.00.

Question **2**

Correct

Mark 2.00 out of 2.00

An e-commerce company plans to give their customers a special discount for Christmas.

They are planning to offer a flat discount. The discount value is calculated as the sum of all

the prime digits in the total bill amount.

Write an algorithm to find the discount value for the given total bill amount.

Constraints

1 <= orderValue< 10e100000

Input

The input consists of an integer orderValue, representing the total bill amount.

Output

Print an integer representing the discount value for the given total bill amount.

Example Input

578

Output

12

**For example:**

| Test | Result |
|---|---|
| print(christmasDiscount(578)) | 12 |

**Answer:** (penalty regime: 0 %)

Reset answer

```
def christmasDiscount(orderValue: int) -> int:
    # Prime single digits are 2, 3, 5, and 7
    # map them to their integer values, non-prime digits map to 0
    prime_map = {
        '2': 2, '3': 3, '5': 5, '7': 7,
        '0': 0, '1': 0, '4': 0, '6': 0, '8': 0, '9': 0
    }

    # Process the value as a string to avoid heavy mathematical iteration
    # on massive digits, summing up the values via the fast hash map
    return sum(prime_map[char] for char in str(orderValue))

```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | print(christmasDiscount(578)) | 12 | 12 | ✓ |
| ✓ | print(christmasDiscount(57)) | 12 | 12 | ✓ |
| ✓ | print(christmasDiscount(222)) | 6 | 6 | ✓ |
| ✓ | print(christmasDiscount(77777)) | 35 | 35 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 2.00/2.00.

Question **3**

Correct

Mark 2.00 out of 2.00

A strobogrammatic number is a number that looks the same when rotated 180 degrees (looked at upside down).

Write a program to determine if a number is strobogrammatic. The number is represented as a string.

**Example 1:**

**Input:**

69

**Output:**

true

**Example 2:**

**Input:**

88

**Output:**

true

**Example 3:**

**Input:**

962

**Output:**

false

**Example 4:**

**Input:**

1

**Output:**

true

**For example:**

| Test | Result |
|---|---|
| `print(Strobogrammatic(69))` | `True` |
| `print(Strobogrammatic(962))` | `False` |

**Answer:** (penalty regime: 0 %)

Reset answer

```
def Strobogrammatic(num) -> bool:
    # Convert input to string to handle integers safely and allow indexing
    num_str = str(num)

    # Map each valid strobogrammatic digit to its 180-degree rotated counterpart
    lookup = {'0': '0', '1': '1', '6': '9', '8': '8', '9': '6'}

    left, right = 0, len(num_str) - 1

    while left <= right:
        # If either character is not invertible, it's not strobogrammatic
        if num_str[left] not in lookup or num_str[right] not in lookup:
            return False

        # The character at the left must rotate to match the character at the right
        if lookup[num_str[left]] != num_str[right]:
            return False

        left += 1
        right -= 1
```

```
21
22      return True
23
```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | print(Strobogrammatic(69)) | True | True | ✓ |
| ✓ | print(Strobogrammatic(88)) | True | True | ✓ |
| ✓ | print(Strobogrammatic(962)) | False | False | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 2.00/2.00.

Question **4**

Correct

Mark 2.00 out of 2.00

A number is considered to be ugly if its only prime factors are 2, 3 or 5.

[1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, ...] is the sequence of ugly numbers.

Task:

complete the function which takes a number n as input and checks if it's an ugly number.

return ugly if it is ugly, else return not ugly

Hint:

An ugly number U can be expressed as: U = 2^a * 3^b * 5^c, where a, b and c are nonnegative integers.

**For example:**

| Test | Result |
|---|---|
| `print(checkUgly(6))` | `ugly` |
| `print(checkUgly(21))` | `not ugly` |

**Answer:** (penalty regime: 0 %)

Reset answer

```
def checkUgly(n: int) -> str:
    # 0 and negative numbers are not ugly numbers
    if n <= 0:
        return "not ugly"

    # Divide out all factors of 2, 3, and 5
    for factor in (2, 3, 5):
        while n % factor == 0:
            n //= factor

    # If the remaining number is 1, it's an ugly number
    return "ugly" if n == 1 else "not ugly"

```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | `print(checkUgly(6))` | `ugly` | `ugly` | ✓ |
| ✓ | `print(checkUgly(21))` | `not ugly` | `not ugly` | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 2.00/2.00.

Question **5**

Correct

Mark 2.00 out of 2.00

Write a function that returns the value of a+aa+aaa+aaaa with a given digit as the value of a.

Suppose the following input is supplied to the program:

9

Then, the output should be:

9+99+999+9999=11106

Sample Input Format:

9

Sample Output format:

11106

**For example:**

| Test | Result |
|---|---|
| print(Summation(8)) | 9872 |

**Answer:** (penalty regime: 0 %)

Reset answer

```
def Summation(a: int) -> int:
    # Convert digit to string to easily create repetitions
    str_a = str(a)

    # Generate the numbers by repeating the digit string
    t1 = int(str_a)
    t2 = int(str_a * 2)
    t3 = int(str_a * 3)
    t4 = int(str_a * 4)

    # Return the final calculated sum
    return t1 + t2 + t3 + t4

```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | print(Summation(8)) | 9872 | 9872 | ✓ |
| ✓ | print(Summation(10)) | 10203040 | 10203040 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 2.00/2.00.